Foundations

relu, step, comparisons, logic, min / max / clamp, and select

---

Before the examples: the ground rules

The idea

A normal program *decides*. It reaches an `if`, follows one branch and never looks at the other. A neural network cannot decide anything: every neuron computes its number every time, and the only operations available are multiply by a weight, add, and one tiny non-linearity, $\mathrm{relu}(x) = \max(0, x)$.

This book series shows that nothing is lost. A program on small integers can be rewritten so that it computes **every** branch and then multiplies the wrong branches by 0. The 0 and the 1 are ordinary numbers (a "mask") produced by the same arithmetic. Written this way, the program is a stack of `relu(W x + b)` layers: a feed-forward network with weights you wrote by hand. Later chapters do this to loops, arrays, BFS, A* and minimax. This chapter builds the six tools everything else is made from.

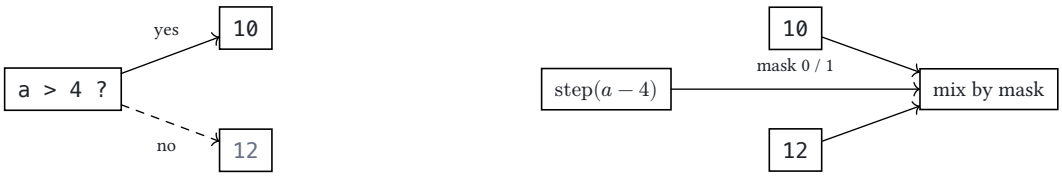


Figure 1: Left: control flow takes one road. Right: data flow takes every road and lets a 0/1 number choose the answer.

Three stages, one rule each

Every example is written three times. The three versions must agree on every input, and each file proves it by running all three on every input in range (or on a fixed random sample plus edge cases when the range is too large).

Stage	What you may write	What you may never write
A · control flow	Normal Python. Anything.	Nothing is forbidden. It is the specification.
B · branchless	Scalar Python: `+`, `−`, multiplying by a **constant**, `relu`, helpers built from those, loops whose count is fixed before the input is seen (`range(N)` with a constant `N`).	`if` on a runtime value, comparisons, `min`/`max`/`abs` on runtime values, multiplying two runtime values, a loop whose length depends on the input.
C · network	numpy: constant-matrix `@`, `+`/`−`, scalar-constant `*`, `np.maximum(0, ·)`, slicing, `np.concatenate`, fixed loops.	The same list, plus `np.where`, `argmax`, `max` on runtime data, and elementwise products of two runtime arrays.

A **runtime value** is anything that depends on the input. A **constant** is known before the input arrives: a literal, `N`, or a weight. The central rule is the one about products: *a weight multiplies a runtime value by a constant, and nothing else multiplies.* Example 06 shows how a mask "multiplies" a runtime value anyway.

Stage B is the algebra: it is where you find the formulas and check them in plain Python. Stage C is the same algebra with the bookkeeping done by matrices: it shows which neurons exist, what they are connected to, and how deep the network is.

Why only integers

All the 0/1 decisions come from one trick (example 01): $\text{step}(x) = \text{relu}(x) - \text{relu}(x-1)$. This is exactly 0 for $x \leq 0$ and exactly 1 for $x \geq 1$, but in between, for $0 < x < 1$, it is a ramp that returns x itself. There is a hole between 0 and 1 where the "decision" is undecided. If every value that can reach step is an **integer**, no input ever lands in the hole, and the ramp is invisible: step is an exact 0/1 switch. That is why every example declares an integer range up front.

Two consequences. First, "$a > b$" is only a statement about integers: the test is $a - b \geq 1$. Second, there is no fuzziness to worry about: small integers are represented exactly by float64, so the files compare A, B and C with plain ==. (If your inputs live on a finer grid, say multiples of $\frac{1}{4}$, scale them by 4 first; example 01 shows it.)

What "a network" means here

A **layer** is the function

$$h = \text{relu}(Wx + b)$$

where x is a vector of numbers coming in, W is a matrix of constants, b is a vector of constants and relu is applied to every entry. **Row i of W together with b_i is one neuron**: it takes a weighted sum of the inputs, adds its bias, and clips at zero. A network is layers stacked, with a final layer that has no relu, only the affine part:

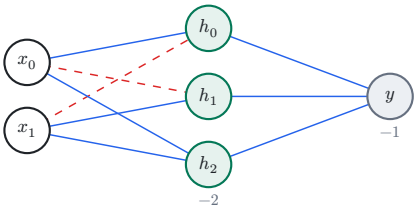
$$y = W_2\,\text{relu}(W_1 x + b_1) + b_2$$

For instance, with $x = (3, 1)$:

$$W_1 x + b_1 = \begin{pmatrix} 1 & -1 \\ -1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ -2 \end{pmatrix} = \begin{pmatrix} 2 \\ -2 \\ 2 \end{pmatrix}$$

$$h = \text{relu}(\ldots) = \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix}$$

$$y = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} + (-1) = 3$$



The diagrams in this booklet draw exactly this. A circle is a neuron; a line is one entry of W (blue: positive, red dashed: negative; a line with no number has weight ± 1; zero entries are not drawn). The number under a neuron is its bias. White circles are inputs, green circles apply relu, grey circles are output neurons that only add.

Stage C is a real feed-forward neural network. If you want to see it with your own eyes, the second example's weights drop straight into any deep-learning library:

```
net = torch.nn.Sequential(torch.nn.Linear(1, 2), torch.nn.ReLU(), torch.nn.Linear(2, 1))
net[0].weight.data = torch.tensor([[1.0], [1.0]]);  net[0].bias.data = torch.tensor([-4.0,
-5.0])
net[2].weight.data = torch.tensor([[-2.0, 2.0]]);   net[2].bias.data = torch.tensor([12.0])
```

Nothing here is trained. Every weight is chosen by reasoning, which is the whole point: the matrices *are* the program, and you can read the program back out of them.

The toolbox

Everything in this chapter is a row of this table. Inputs are integers where step appears.

Want	Data-flow formula	Holds when	Example
$x \geq 1$ as 0/1	$\text{step}(x) = \text{relu}(x) - \text{relu}(x-1)$	x integer	01
`a > b`	$\text{step}(a-b)$	integers	03
`a >= b`	$\text{step}(a-b+1)$	integers	03
`a == b`	$1 - \text{step}(a-b) - \text{step}(b-a)$	integers	03
`not p`	$1-p$	$p \in \{0,1\}$	04
`p and q`	$\text{relu}(p+q-1)$	bits	04
`p or q`	$\text{step}(p+q)$	bits	04
`max(a, b)`	$b + \text{relu}(a-b)$	always	05
`min(a, b)`	$a - \text{relu}(a-b)$	always	05
`abs(x)`	$\text{relu}(x) + \text{relu}(-x)$	always	05
`clamp(x, lo, hi)`	$\text{lo} + \text{relu}(x-\text{lo}) - \text{relu}(x-\text{hi})$	$\text{lo} \leq \text{hi}$	05
`x if m else 0`	$\text{relu}(x - M(1-m)) - \text{relu}(-x - M(1-m))$	$m \in \{0,1\}$, $\lvert x\rvert \leq M$	06
`x if m else y`	$y + \text{select}(m, x-y)$	same, $\lvert x-y\rvert \leq M$	06

01 relu and step

`basic_python/01_foundations/01_relu_and_step.py`

The problem

Write `1 if x >= 1 else 0` for an integer `x` without an `if`, using only $\text{relu}(x) = \max(0, x)$ and arithmetic. This one gadget turns a number into a 0/1 switch, and every later example is built from it.

relu, then step

relu is flat at 0 for negative input and the line $y = x$ for positive input: it is the identity with the negative half chopped off. Slide it right by one, $\text{relu}(x - 1)$, and the kink moves from $x = 0$ to $x = 1$. Subtract the second from the first and the slopes cancel everywhere the two lines are both sloped: $x - (x - 1) = 1$.

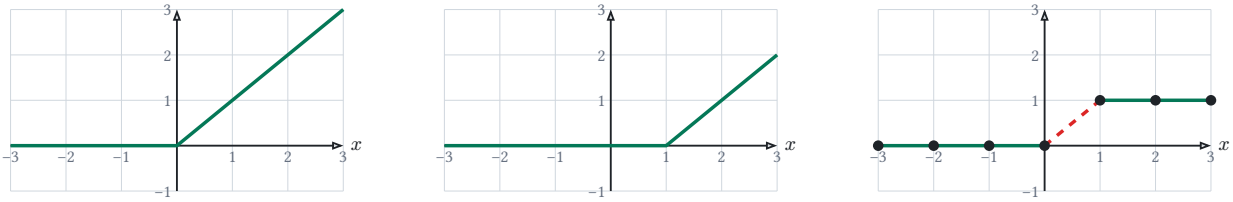


Figure 2: Left to right: $\text{relu}(x)$, $\text{relu}(x - 1)$, and their difference $\text{step}(x) = \text{relu}(x) - \text{relu}(x - 1)$. The black dots are the integers; the dashed red ramp between 0 and 1 is the only place the formula is not 0 or 1, and no integer sits on it.

A · control flow

```python
def control_flow(x):
    return 1 if x >= 1 else 0
```

B · branchless arithmetic

```python
def relu(x):
    return max(0, x)  # the primitive

def branchless(x):
    return relu(x) - relu(x - 1)
```

`relu` is the one place `max` is allowed: it is the building block we are given, not a comparison we smuggled in. Everything else in stage B is `+`, `-` and calls to `relu`.

Why it works: a table, then a proof

x	−3	−2	−1	0	1	2	3	4
$\text{relu}(x)$	0	0	0	0	1	2	3	4
$\text{relu}(x - 1)$	0	0	0	0	0	1	2	3
$\text{step}(x)$	0	0	0	0	1	1	1	1

Proof by cases. If $x \leq 0$, both relus return 0, so the difference is $0 - 0 = 0$. If $x \geq 1$, both are on their sloped part, so the difference is $x - (x - 1) = 1$. The only case left is $0 < x < 1$, which contains no integer. So on integers, $\text{step}(x) = 1$ exactly when $x \geq 1$.

Moving the threshold. To test $x \geq k$, evaluate $\text{step}(x - k + 1)$. To test $a > 4$, that is $a - 4 \geq 1$, evaluate $\text{step}(a - 4)$: this is exactly what example 02 does.

What happens at 0.5 (the reason for "integers")

Feed the same formula numbers that are not integers:

x	−0.5	0	0.25	0.5	0.75	1	1.5
$\text{relu}(x)$	0	0	0.25	0.5	0.75	1	1.5
$\text{relu}(x - 1)$	0	0	0	0	0	0	0.5
$\text{step}(x)$	0	0	0.25	0.5	0.75	1	1

Between 0 and 1 the "switch" returns x itself: $\text{step}(0.5) = 0.5$, neither "no" nor "yes". Arithmetic on its own cannot repair this: relu is continuous, so any combination of relus is continuous and cannot jump from 0 to 1 at a point. With these unit weights the ramp is exactly one unit wide, and the cure is to make sure inputs never land inside it. Integers guarantee that, because the gap between consecutive integers is exactly the width of the ramp.

If your values live on a finer grid, rescale instead. For multiples of $\frac{1}{4}$, multiply by 4 (the weights become 4 instead of 1): $\text{relu}(4x) - \text{relu}(4x - 1)$ is 1 exactly when $x \geq \frac{1}{4}$. The ramp is now only $\frac{1}{4}$ wide and fits between grid points. `01_relu_and_step.py` prints both tables.

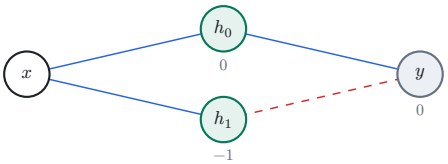
Stage C: the smallest interesting network

One input, two hidden neurons, one output. Hidden neuron 0 computes $\text{relu}(x)$ (weight 1, bias 0); hidden neuron 1 computes $\text{relu}(x - 1)$ (weight 1, bias -1); the output neuron subtracts them.

$$W_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

$$W_2 = (1 \ \ -1), \quad b_2 = (0)$$

$$y = W_2 \, \text{relu}(W_1 x + b_1) + b_2$$



Worked trace, one column per input:

	$x = -2$	$x = 0$	$x = 1$	$x = 3$
$W_1 x + b_1$	$(-2, -3)$	$(0, -1)$	$(1, 0)$	$(3, 2)$
$h = \text{relu}(\ldots)$	$(0, 0)$	$(0, 0)$	$(1, 0)$	$(3, 2)$
$y = h_0 - h_1$	0	0	1	1

Look at the last column: neuron 0 reports 3, neuron 1 reports 2, and the output $3 - 2 = 1$ is independent of how large x is. That is a **saturating** unit: the output stops growing because the second relu "catches up" with the first. We will use that shape constantly.

What to notice

- **Where the 0/1 comes from.** Nothing in the network is a threshold. The "jump" is the difference of two slopes cancelling. Two neurons per step.
- **Cost:** 2 neurons, 1 hidden layer. Every comparison later costs this much (or shares it).
- **Pitfall:** feeding step a non-integer. The code prints 0.25 and 0.5 without complaint. Garbage in, silent garbage out.
- **Pitfall:** off-by-one in the threshold. $\text{step}(x)$ means $x \geq 1$, not $x > 0$ (these only agree on integers).

> **Try it**
>
> Build `1 if x >= 3 else 0` as a network with 1 input, 2 hidden neurons and 1 output. Which weights and biases change compared to W_1, b_1, W_2, b_2 above? (Answer in the last section.)

02 if / else

basic_python/01_foundations/02_if_else.py

The problem

```
result = 10 if a > 4 else 12        # a is an integer in [-20, 20]
```

Control flow runs one branch and skips the other. Data flow cannot skip: it evaluates **both** branches (here they are the constants 10 and 12) and turns the condition into a 0/1 number that says how much of each to keep.

A · control flow

```
def control_flow(a):
    if a > 4:
        return 10
    else:
        return 12
```

B · branchless arithmetic

```
def branchless(a):
    above4 = step(a - 4)
    return above4 * 10 + (1 - above4) * 12
```

Derivation

1. **The condition becomes a mask.** For integers, $a > 4$ means $a - 4 \geq 1$, which is exactly what step tests: $m = \text{step}(a - 4)$. The mask is 1 when the condition holds and 0 otherwise.
2. **Both branches are weighted.** $m \cdot 10 + (1 - m) \cdot 12$. This looks like multiplying two runtime values, but 10 and 12 are constants, so $m \cdot 10$ is "multiply a runtime value by a constant": allowed. If the branches were runtime values, we would have a problem; that is example 06.
3. **Simplify.** $m \cdot 10 + (1 - m) \cdot 12 = 12 - 2m$: an affine function of the mask.
4. **Expand the step.** $12 - 2(\text{relu}(a - 4) - \text{relu}(a - 5)) = 12 - 2\,\text{relu}(a - 4) + 2\,\text{relu}(a - 5)$. Now it is "constant, plus weights times relus": exactly the shape of a network.

a	2	3	4	5	6	7	8
$\text{relu}(a - 4)$	0	0	0	1	2	3	4
$\text{relu}(a - 5)$	0	0	0	0	1	2	3
mask $m = \text{step}(a - 4)$	0	0	0	1	1	1	1
$12 - 2m$	12	12	12	10	10	10	10

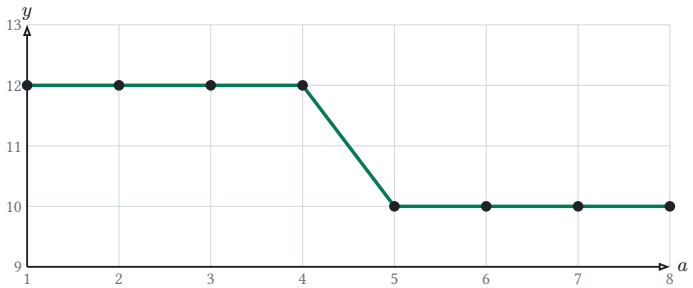


Figure 3: The function the network computes, for real a (curve) and for integer a (dots). The one-unit ramp between 4 and 5 is the same ramp as in example 01; no integer lands on it.

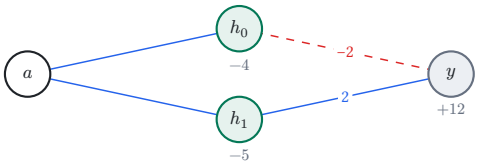
Stage C: the network

Read the formula $12 - 2\,\text{relu}(a - 4) + 2\,\text{relu}(a - 5)$ left to right and it already is a network. Two hidden neurons, one per relu; their incoming weights are 1, their biases are the -4 and -5 inside; the output neuron has weights -2, $+2$ and bias 12.

$$W_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} -4 \\ -5 \end{pmatrix}$$

$$W_2 = (-2 \;\; 2), \quad b_2 = (12)$$

$$y = W_2 \,\mathrm{relu}(W_1 a + b_1) + b_2$$



Worked trace for $a = 7$ (condition true, expected 10):

$$W_1 a + b_1 = \begin{pmatrix} 7 \\ 7 \end{pmatrix} + \begin{pmatrix} -4 \\ -5 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix} \implies h = \begin{pmatrix} 3 \\ 2 \end{pmatrix} \implies y = (-2)(3) + 2(2) + 12 = 10.$$

and for $a = 4$ (condition false, expected 12):

$$W_1 a + b_1 = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \implies h = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies y = 0 + 0 + 12 = 12.$$

and for $a = 5$, the first value above the threshold: $h = (1, 0)$, so $y = -2 + 0 + 12 = 10$.

Key idea

Below the threshold both neurons are silent (0), so the output is the bias: the "else" value 12. Above it both fire and their difference saturates at 1, so the output is $12 - 2 = 10$, the "then" value. The bias holds the else-branch; the weights hold the difference (then − else).

What to notice

- **The general pattern.** For constants X (then) and Y (else): result $= Y + (X - Y) \cdot \mathrm{step}(\mathrm{test})$. Both branch values go into weights and the bias. The branch **code** has already run on every input; the mask only selects its result.
- **`>` versus `>=`.** $\mathrm{step}(a - 4)$ is $a > 4$. For $a \geq 4$ use $\mathrm{step}(a - 3)$. Shift the thresholds, not the formula. Off-by-one here is the most common mistake.
- **No range needed.** The weights never mention $[-20, 20]$: the branches are constants, so nothing can overflow. (Contrast with example 06, where the branches are runtime numbers and need a bound M.)
- **Cost:** 1 hidden layer, 2 neurons per `if`. Chaining `elif`s adds 2 neurons each (see the exercise).

Try it

Turn `1 if a < 0 else 5 if a < 10 else 9` into one hidden layer. How many neurons, and what is the output formula? (Answer at the end.)

03 comparisons as numbers

basic_python/01_foundations/03_compare.py

The problem

Produce all six comparisons of two integers, `a > b`, `a >= b`, `a < b`, `a <= b`, `a == b`, `a != b`, each as a 0/1 **number** that later arithmetic can use (as a mask, as a count, as a multiplier). Here $a, b \in [-8, 8]$.

A · control flow

```python
def control_flow(a, b):
    return (int(a > b), int(a >= b),
            int(a < b), int(a <= b),
            int(a == b), int(a != b))
```

B · branchless arithmetic

```python
def branchless(a, b):
    d = a - b
    gt = step(d)
    ge = step(d + 1)
    lt = step(-d)
    le = step(-d + 1)
    eq = 1 - gt - lt
    ne = gt + lt
    return (gt, ge, lt, le, eq, ne)
```

Derivation

Only the difference $d = a - b$ matters, and for integers every comparison is a threshold on d (or on $-d$):

Test	As a threshold	Formula
`a > b`	$d \geq 1$	$\text{step}(d)$
`a >= b`	$d \geq 0$, i.e. $d + 1 \geq 1$	$\text{step}(d + 1)$
`a < b`	$-d \geq 1$	$\text{step}(-d)$
`a <= b`	$-d \geq 0$, i.e. $-d + 1 \geq 1$	$\text{step}(-d + 1)$
`a == b`	neither > nor <	$1 - \text{step}(d) - \text{step}(-d)$
`a != b`	> or < (never both)	$\text{step}(d) + \text{step}(-d)$

Two ideas do all the work. **Shifting** the argument of step by one turns > into >=, because step tests "≥ 1" and shifting by one moves the threshold from 1 to 0. **Negating** the argument mirrors the comparison. For == and !=, the two strict inequalities cannot both be true and cannot both be false when $a = b$, so adding them counts "not equal" and subtracting from 1 gives "equal". Here is every value, with d running over a few integers:

$d = a - b$	−3	−2	−1	0	1	2	3
`a > b`	0	0	0	0	1	1	1
`a >= b`	0	0	0	1	1	1	1
`a < b`	1	1	1	0	0	0	0
`a <= b`	1	1	1	1	0	0	0
`a == b`	0	0	0	1	0	0	0
`a != b`	1	1	1	0	1	1	1

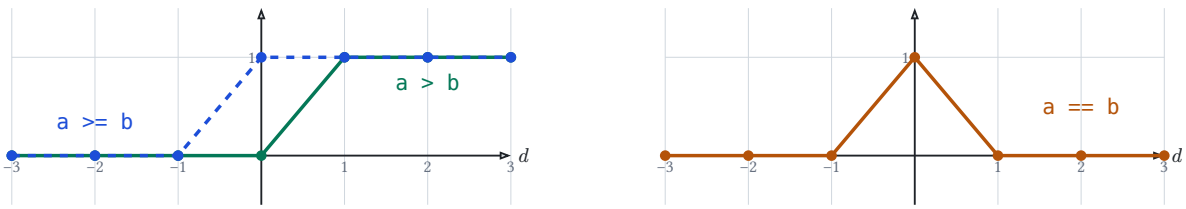


Figure 4: Left: `a > b` (solid) and `a >= b` (dashed) as functions of $d = a - b$; the second is the first shifted left by one. Right: `a == b` is a "tent" of height 1 and width 2 around $d = 0$; only integers d (dots) are ever fed in.

Stage C: six shared neurons

Write each step out as two relus. The four steps use eight relus, but neighbouring ones coincide: $\text{step}(d) = \text{relu}(d) - \text{relu}(d-1)$ and $\text{step}(d+1) = \text{relu}(d+1) - \text{relu}(d)$ share $\text{relu}(d)$. That leaves six hidden neurons, three on d and three on $-d$:

$$h_0 = \text{relu}(d+1), \quad h_1 = \text{relu}(d), \quad h_2 = \text{relu}(d-1), \quad h_3 = \text{relu}(-d+1), \quad h_4 = \text{relu}(-d), \quad h_5 = \text{relu}(-d-1)$$

	h_0	h_1	h_2	h_3	h_4	h_5
`a > b`		$+$	$-$			
`a >= b`	$+$	$-$				
`a < b`					$+$	$-$
`a <= b`				$+$	$-$	
`a == b`		$-$	$+$		$-$	$+$
`a != b`		$+$	$-$		$+$	$-$

This table *is* the output matrix W_2. Each comparison picks two neighbouring neurons of the same family (that is a step), and `==`/`!=` simply add or subtract the `>` and `<` rows (`==` also adds the constant 1).

$$W_1 = \begin{pmatrix} 1 & -1 \\ 1 & -1 \\ 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ -1 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 1 \\ 0 \\ -1 \\ 1 \\ 0 \\ -1 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 0 & 1 & -1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 & -1 & 0 \\ 0 & -1 & 1 & 0 & -1 & 1 \\ 0 & 1 & -1 & 0 & 1 & -1 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

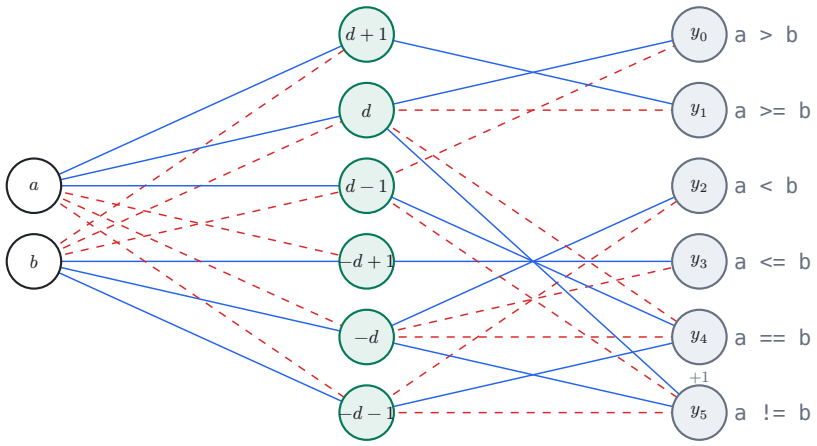


Figure 5: Two inputs, one hidden layer of six relu neurons (each labelled with the number it computes from $d = a - b$), six outputs. The biases of the hidden layer are folded into the labels: $+1, 0, -1, +1, 0, -1$.

Worked trace. Each column is one input pair.

	$(a, b) = (5, 3)$	$(4, 4)$	$(3, 5)$
d	2	0	-2
$W_1 x + b_1$	$(3, 2, 1, -1, -2, -3)$	$(1, 0, -1, 1, 0, -1)$	$(-1, -2, -3, 3, 2, 1)$
h	$(3, 2, 1, 0, 0, 0)$	$(1, 0, 0, 1, 0, 0)$	$(0, 0, 0, 3, 2, 1)$
`> >= < <= == !=`	$1, 1, 0, 0, 0, 1$	$0, 1, 0, 1, 1, 0$	$0, 0, 1, 1, 0, 1$

For $(5, 3)$: `a > b` $= h_1 - h_2 = 2 - 1 = 1$; `a >= b` $= h_0 - h_1 = 3 - 2 = 1$; `a < b` $= h_4 - h_5 = 0$; `a == b` $= 1 - 1 - 0 = 0$.

What to notice

- **Sharing is free.** Neurons are only worth counting once: 6 neurons produce six answers. If a later network needs several comparisons of the same difference, it should reuse $h_0, \ldots, h_5$ rather than adding more.
- **A shorter route to ==.** `a == b` is `a >= b` minus `a > b`, which equals $\mathrm{relu}(d + 1) - 2\,\mathrm{relu}(d) + \mathrm{relu}(d - 1)$: a tent made of three neurons (the right-hand plot). The file uses $1 - \mathrm{gt} - \mathrm{lt}$ instead because it reads straight off the toolbox; both are right.
- **Integers only.** For $a = 0.5$, $b = 0$: `a > b` gives 0.5 and `a == b` gives 0.5. There is no warning: the formulas return numbers between 0 and 1.
- **Mirroring.** `a < b` is `b > a`: swap the inputs and the same neurons apply. In the network the swap is the sign flip in rows 3–5 of W_1.
- **Cost:** 1 hidden layer, 6 neurons, for any comparison of any range of integers (no bound needed).

> **Try it**
>
> Build `abs(a - b) <= 2` ("a and b are close") in a single hidden layer. How many neurons? (Answer at the end.)

04 logic on bits

basic_python/01_foundations/04_logic.py

The problem

Compute `not p`, `p and q`, `p or q`, `p xor q` and `p implies q` for two bits $p, q \in \{0, 1\}$ (all four inputs are checked), plus `all(bits)` and `any(bits)` for four bits (all sixteen inputs). In data flow a bit is just the number 0 or 1, so the truth tables become arithmetic.

A · control flow

```python
def control_flow(p, q):
    return (int(not p), int(p and q),
            int(p or q), int(p != q),
            int((not p) or q))
```

B · branchless arithmetic

```python
def branchless(p, q):
    s = p + q
    not_p = 1 - p
    or_ = step(s)
    and_ = relu(s - 1)
    nand = 1 - and_
    xor = relu(or_ + nand - 1)
    implies = 1 - relu(p - q)
    return (not_p, and_, or_,
            xor, implies)
```

Derivation: everything is a function of the sum

For bits, $s = p + q$ can only be 0, 1 or 2, and AND, OR, XOR do not care *which* bit was set, only how many:

p	q	s	not p	p and q	p or q	nand	p xor q	p -> q	$p - q$
0	0	0	1	0	0	1	0	1	0
0	1	1	1	0	1	1	1	1	−1
1	0	1	0	0	1	1	1	0	1
1	1	2	0	1	1	0	0	1	0

1. **AND** is 1 only when $s = 2$: $\text{relu}(s - 1)$ is $0, 0, 1$ for $s = 0, 1, 2$.
2. **OR** is 1 when $s \geq 1$: $\text{step}(s)$, or equivalently $\text{relu}(s) - \text{relu}(s - 1)$.
3. **NOT** is $1 - p$: it flips 0 and 1 and needs no relu.
4. **IMPLIES** ($p \to q$, "if p then q") is false only for $p = 1, q = 0$, i.e. when $p - q = 1$. So $1 - \text{relu}(p - q)$: $p - q$ is $-1, 0, 0, 1$ and only the last survives the relu. One neuron.

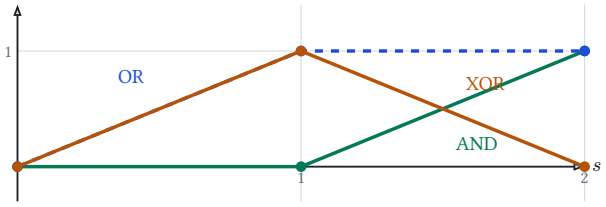


Figure 6: AND, OR and XOR of two bits, as functions of the sum s. AND and OR are "monotone": once s is big enough the answer stays 1. XOR goes up *and back down*.

XOR is different, and why two layers helped historically

AND and OR are "s above a threshold". XOR is "s is between two thresholds". No single line in the (p, q) plane separates the XOR-true corners from the XOR-false ones: it takes two parallel lines, one for OR (s = 0.5) and one for AND (s = 1.5), with the true corners in the band between.

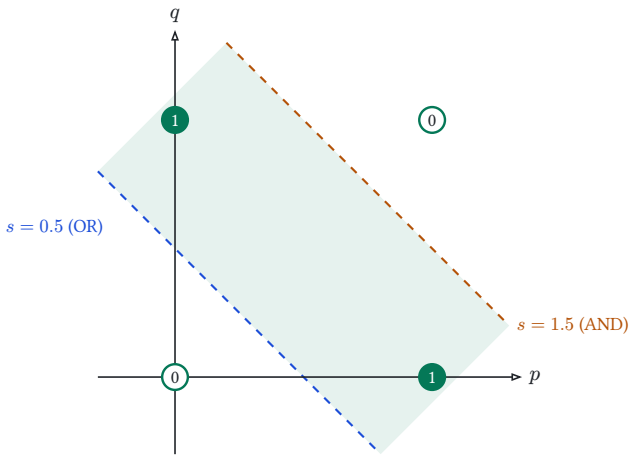


Figure 7: Truth values of XOR at the four corners of the (p, q) square. The two dashed lines are the thresholds of OR and AND; XOR is true in the shaded band between them.

The classic construction (older than ReLU) names the band's two edges and combines them with a second layer:

$$p \oplus q = (p \vee q) \wedge \neg(p \wedge q) = \mathrm{AND}(\mathrm{OR}(p, q), \mathrm{NAND}(p, q))$$

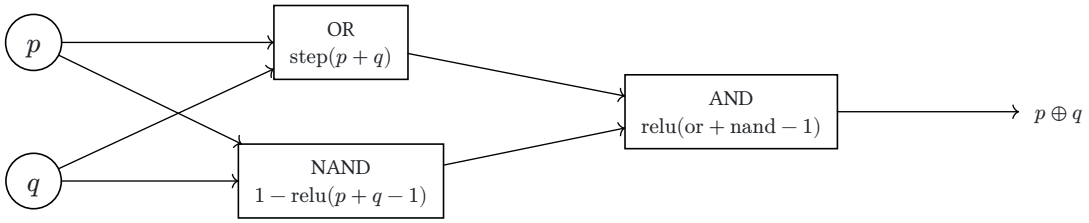


Figure 8: Layer 1 names the band's edges (OR and NAND are both threshold units); layer 2 ANDs them.

Check with $p = q = 1$: or $= 1$, nand $= 1 - 1 = 0$, $\mathrm{relu}(1 + 0 - 1) = 0$. Check with $p = 1, q = 0$: or $= 1$, nand $= 1$, $\mathrm{relu}(1 + 1 - 1) = 1$.

Stage C: a two-hidden-layer network

The outputs of a layer are *only* what the next layer can see, so every number needed later must be present in the previous layer's output. Two rules make that work:

- **Copy trick.** If $v \geq 0$ then $\mathrm{relu}(v) = v$, so a hidden neuron with weight 1 and bias 0 on a non-negative value copies it forward. Bits, sums and the relus themselves are all non-negative.
- **Differences are free.** or $= \mathrm{relu}(s) - \mathrm{relu}(s - 1)$ is not a neuron; it is a weighted sum of two neurons, which the next layer takes as input.

Layer 1 has four neurons, $x = (p, q)$ and $s = p + q$: $h_0 = \mathrm{relu}(s)$, $h_1 = \mathrm{relu}(s - 1)$ (this is AND), $h_2 = \mathrm{relu}(p - q)$ (p and not q) and $h_3 = \mathrm{relu}(p)$ (a copy of p). Layer 2 has five: $g_0 = \mathrm{relu}(h_0 - 2h_1)$ is XOR, because or + nand − $1 = (h_0 - h_1) + (1 - h_1) - 1 = h_0 - 2h_1$; the other four copy or, and, $p \wedge \neg q$ and p forward. The last layer only reorders them and applies $1 - \cdot$ for `not p` and `p -> q`.

$$W_1 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & -1 \\ 1 & 0 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1 \\ 0 \\ 0 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 1 & -2 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$W_3 = \begin{pmatrix} 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 \end{pmatrix}, \quad b_3 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

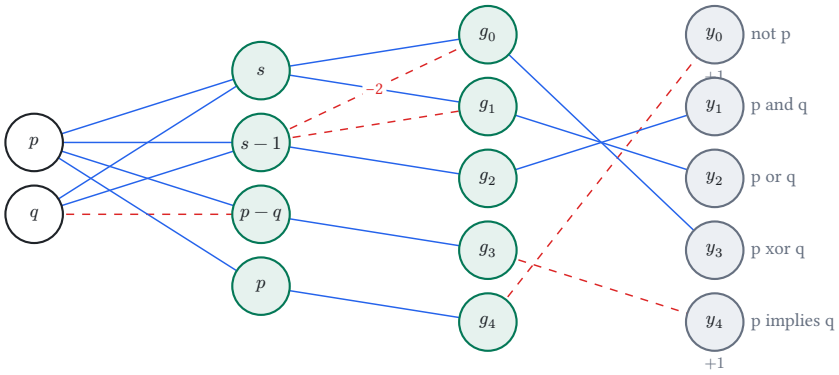


Figure 9: Layer 1 (labelled by the quantity inside its relu) holds s, AND, "p and not q" and p. Layer 2: g_0 = XOR; $g_1, \ldots, g_4$ copy OR, AND, "p and not q" and p forward. The output layer only reorders and negates.

Worked trace for $(p, q) = (1, 0)$ and $(1, 1)$:

	$(p,q) = (1,0)$	$(1,1)$
$W_1 x + b_1$	$(1, 0, 1, 1)$	$(2, 1, 0, 1)$
h	$(1, 0, 1, 1)$	$(2, 1, 0, 1)$
$W_2 h + b_2$	$(1, 1, 0, 1, 1)$	$(0, 1, 1, 0, 1)$
g	$(1, 1, 0, 1, 1)$	$(0, 1, 1, 0, 1)$
y (not, and, or, xor, implies)	$(0, 0, 1, 1, 0)$	$(0, 1, 1, 0, 1)$

Shallower: XOR in one hidden layer

Because $\text{or} = \text{relu}(s) - \text{relu}(s-1)$ and $\text{and} = \text{relu}(s-1)$, XOR is a plain weighted sum of layer 1:

$$p \oplus q = \text{or} - \text{and} = \text{relu}(s) - 2\,\text{relu}(s-1).$$

Check: $s = 0 \to 0$; $s = 1 \to 1 - 0 = 1$; $s = 2 \to 2 - 2 = 0$. The tent of the plot above. So with the same four hidden neurons and a different output matrix the network needs only one hidden layer (`network_shallow` in the file):

$$W_{\text{out}} = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}, \quad b_{\text{out}} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

This works because a ReLU "threshold" is a difference of two neurons, not one: a single ReLU neuron really can't do XOR (it is monotone in s), but two of them in one layer can. The two-layer version is still worth knowing: it is the pattern for *any* function of several decisions, where layer 2 reads the 0/1 decisions produced by layer 1.

n bits

For n bits let s be their sum, from 0 to n. All bits are 1 iff $s = n$; at least one is 1 iff $s \geq 1$:

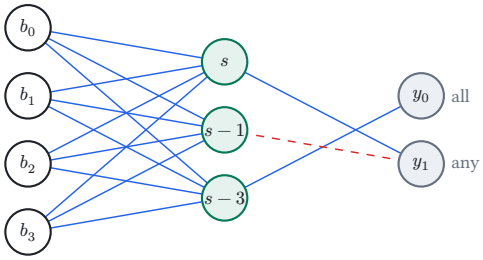
$$\text{AND}(b_1, \ldots, b_n) = \text{relu}(s - (n-1)), \quad \text{OR}(b_1, \ldots, b_n) = \text{step}(s) = \text{relu}(s) - \text{relu}(s-1).$$

More generally "at least k of the n bits are 1" is $\text{step}(s - k + 1)$: $k = 1$ is OR, $k = n$ is AND, $k = 2$ with $n = 3$ is the majority. For $n = 4$:

s	0	1	2	3	4
$\mathrm{relu}(s-3)$ = AND	0	0	0	0	1
$\mathrm{step}(s)$ = OR	0	1	1	1	1

$$W_1 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1 \\ -3 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & -1 & 0 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$



In stage B the sum is `s = 0; for i in range(N): s = s + bits[i]`: the loop count `N` is a constant, not data.

What to notice

- **Bits must be exactly 0 or 1.** With $p = 2$ the formula for `not p` returns -1 and `p and q` is no longer a bit. These formulas are only correct on their domain, and nothing in the network checks the domain.
- **AND and OR are one neuron-pair; XOR is the first function that needs a "band".** More generally, any function of the sum that goes up and comes down needs more than two neurons.
- **Depth versus width.** The deep XOR (3 layers of weights) and the shallow XOR (2) compute the same function. Depth lets later layers reuse earlier decisions; width lets you flatten it. Both are fine for us; later chapters use depth freely because each loop iteration is another layer.
- **Pass-through costs neurons.** Copying a value across a layer costs one neuron per value (when it is ≥ 0). The deep network above needs 4 copy neurons for that.

> **Try it**
>
> `majority(b0, b1, b2)` is 1 when at least two of three bits are 1. Give the formula and the hidden neurons (one hidden layer). Then say what changes for "at least 3 of 5". (Answer at the end.)

05 max, min, abs, clamp, and stacking them

`basic_python/01_foundations/05_min_max_abs_clamp.py`

The problem

Part 1: compute `max(a, b)`, `min(a, b)`, `abs(a - b)` and `clamp(a, -3, 4)` in one network, for integers $a, b \in [-8, 8]$. Part 2: the median of three numbers, $a, b, c \in [-6, 6]$, to see what happens when helpers are *nested*: that is where depth comes from.

A · control flow

```
def control_flow(a, b):
    return (max(a, b), min(a, b),
            abs(a - b),
            min(max(a, LO), HI))

def median3_control_flow(a, b, c):
    return sorted([a, b, c])[1]
```

B · branchless arithmetic

```
def max2(a, b): return b + relu(a - b)
def min2(a, b): return a - relu(a - b)
def abs1(x):    return relu(x) + relu(-x)
def clamp(x, lo, hi):
    return lo + relu(x - lo) - relu(x -
hi)

def median3_branchless(a, b, c):
    return max2(min2(a, b),
                min2(max2(a, b), c))
```

Derivation of the four helpers

All four are built from a single relu of a difference, so none of them uses step. They are exact for any numbers, not just integers.

1. **max.** $\max(a, b) = b + \text{relu}(a - b)$. If $a \geq b$ the relu returns $a - b$ and the sum is a. If $a < b$ it returns 0 and the sum is b. Geometrically: start at b, and "go up by the amount a exceeds b, if it does".
2. **min.** $\min(a, b) = a - \text{relu}(a - b)$. If $a \geq b$ we subtract the excess and land on b; otherwise a stays. (Check: $\max + \min = a + b$ always.)
3. **abs.** $|x| = \text{relu}(x) + \text{relu}(-x)$. Exactly one of the two is non-zero, and it is $|x|$.
4. **clamp.** $\text{clamp}(x, \text{lo}, \text{hi}) = \text{lo} + \text{relu}(x - \text{lo}) - \text{relu}(x - \text{hi})$, valid for $\text{lo} \leq \text{hi}$. Below lo both relus are 0, so the result is lo. Between them the first relu is $x - \text{lo}$ and the second is 0, so the result is x. Above hi the second relu starts subtracting at slope 1 and cancels the first one's slope exactly, freezing the result at $\text{lo} + (\text{hi} - \text{lo}) = \text{hi}$. (This is the same shape as step: a "ramp", then flat. In fact $\text{step} = \text{clamp}(x, 0, 1)$.)

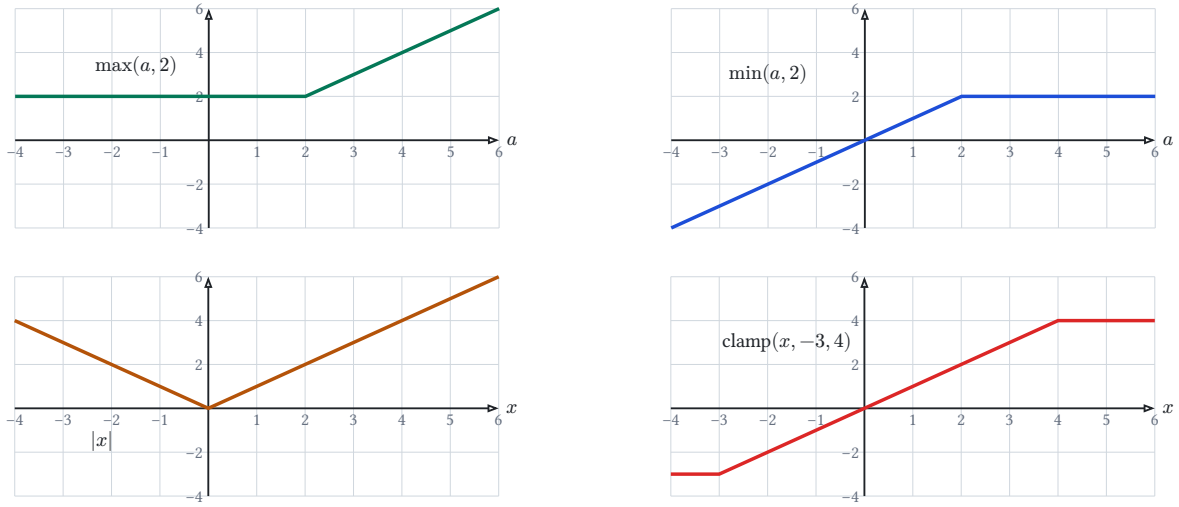


Figure 10: The four helpers as functions of one variable (the other fixed at $b = 2$ for max and min). Each kink is a relu: one for max, min and abs; two for clamp.

Stage C part 1: one hidden layer, with "carried" values

The network sees $x = (a, b)$ and must output four numbers. Notice what the formulas need: $\max = b + \text{relu}(a - b)$ contains a bare b, and min contains a bare a. The output layer only sees hidden neurons, never the input itself, so b and a have to be *carried* through the hidden layer. The identity that carries any signed value:

$$x = \text{relu}(x) - \text{relu}(-x)$$

costs two neurons per value (one if the value is known to be non-negative; see part 2 for a cheaper trick). So eight hidden neurons:

neuron	computes	used for
h_0, h_1	$\text{relu}(a-b)$, $\text{relu}(b-a)$	h_0: max, min, $\lvert a-b\rvert$. h_1: only $\lvert a-b\rvert$
h_2, h_3	$\text{relu}(a)$, $\text{relu}(-a)$	carry $a = h_2 - h_3$, for min
h_4, h_5	$\text{relu}(b)$, $\text{relu}(-b)$	carry $b = h_4 - h_5$, for max
h_6, h_7	$\text{relu}(a+3)$, $\text{relu}(a-4)$	clamp: $-3 + h_6 - h_7$

$$W_1 = \begin{pmatrix} 1 & -1 \\ -1 & 1 \\ 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 3 \\ -4 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ -1 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ -3 \end{pmatrix}$$

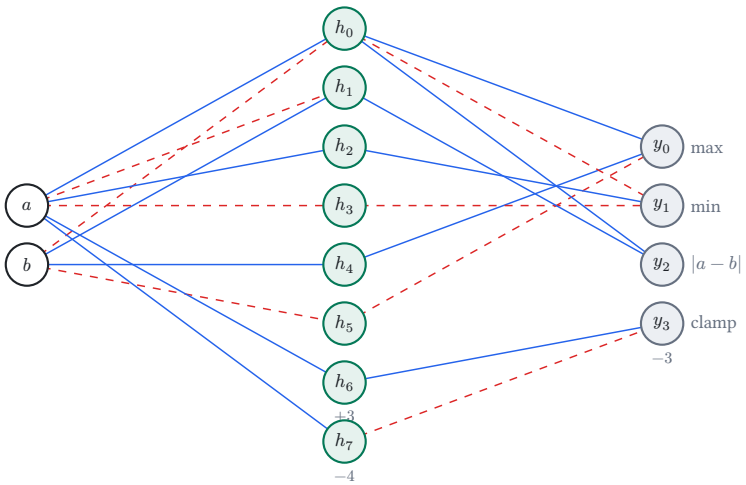


Figure 11: Part 1 network. h_0 is shared by three outputs; the pairs (h_2, h_3) and (h_4, h_5) only carry a and b to the output.

Worked trace, $(a, b) = (7, -2)$:

$W_1 x + b_1$	$(9, -9, 7, -7, -2, 2, 10, 3)$
$h = \text{relu}(\ldots)$	$(9, 0, 7, 0, 0, 2, 10, 3)$
$\max = h_0 + h_4 - h_5$	$9 + 0 - 2 = 7$
$\min = -h_0 + h_2 - h_3$	$-9 + 7 - 0 = -2$
$\lvert a - b \rvert = h_0 + h_1$	$9 + 0 = 9$
$\text{clamp} = -3 + h_6 - h_7$	$-3 + 10 - 3 = 4$

Part 2: composition is stacking layers

The median of three is "the one in the middle": drop the smallest and the largest. With helpers, in the order a human would write it:

$$\text{median}(a, b, c) = \max(\min(a, b), \min(\max(a, b), c))$$

Each helper is one relu, and the inner helpers feed the outer ones. Name what each helper produces:

step	quantity	depends on
1	$u = \text{relu}(a - b)$, so $\text{hi} = \max(a, b) = b + u$, $\text{lo} = \min(a, b) = a - u$	inputs
2	$v = \text{relu}(\text{hi} - c)$, so $\text{mid} = \min(\text{hi}, c) = \text{hi} - v$	step 1
3	$w = \text{relu}(\text{lo} - \text{mid})$, so $\text{median} = \text{mid} + w$	steps 1, 2

A relu that takes another relu as input is a second layer. The nesting $u \to v \to w$ is three relus deep, so the network has **three hidden layers**, no matter how it is arranged. That is the rule: **composition of helpers = stacking layers; depth equals the nesting depth of relus.** Two simplifications make the arithmetic leaner. First, $\text{lo} - \text{mid} = \text{lo} - \text{hi} + v = v - d$ where $d = \text{hi} - \text{lo} = \lvert a - b \rvert = u + u'$ and $u' = \text{relu}(b - a)$ (so $w = \text{relu}(v - d)$). Second, $\text{median} = \text{hi} - v + w$.

Carrying with an offset. Everything a later layer needs must exist in the previous layer. We need b (for $\text{hi} = b + u$) and c (for v) in layer 2. A pair $\text{relu}(x) - \text{relu}(-x)$ would cost two neurons each; instead, since $a, b, c \in [-S, S]$ with $S = 6$, shift: $\text{relu}(b + S) = b + S$ is one neuron and never clips. The offset is paid back in the bias of the last layer. Write $B' = b + S$, $C' = c + S$, $H' = \text{hi} + S$, $m' = \text{mid} + S$.

layer 1	$u = \text{relu}(a - b)$, $u' = \text{relu}(b - a)$, $B' = \text{relu}(b + S)$, $C' = \text{relu}(c + S)$
layer 2	$v = \text{relu}(B' + u - C')$ $(= \text{relu}(\text{hi} - c))$, $d = \text{relu}(u + u')$ $(= \lvert a - b \rvert)$, $H' = \text{relu}(B' + u)$ $(= \text{hi} + S)$
layer 3	$w = \text{relu}(v - d)$, $m' = \text{relu}(H' - v)$ $(= \text{mid} + S)$
output	$y = m' + w - S$ $(= \text{mid} + w = \text{median})$

$$W_1 = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ 0 \\ 6 \\ 6 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

$$W_3 = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 0 & 1 \end{pmatrix}, \quad b_3 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad W_4 = (1 \;\; 1), \quad b_4 = (-6)$$

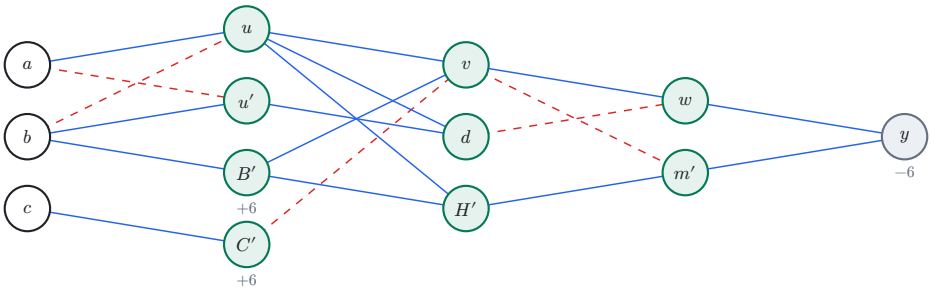


Figure 12: Median of three: 3 inputs, hidden layers of 4, 3 and 2 neurons, one output. Three relus deep because the nesting is three deep.

Worked traces:

(a, b, c)	layer 1 (u, u', B', C')	layer 2 (v, d, H')	layer 3 $(w, m') \to y$
$(4, 5, -2)$	$(0, 1, 11, 4)$	$(7, 1, 11)$	$(6, 4) \to 6 + 4 - 6 = 4$
$(1, 5, 3)$	$(0, 4, 11, 9)$	$(2, 4, 11)$	$(0, 9) \to 0 + 9 - 6 = 3$
$(5, 1, 3)$	$(4, 0, 7, 9)$	$(2, 4, 11)$	$(0, 9) \to 3$

In $(4, 5, -2)$: $\text{hi} = 5, \text{lo} = 4, c = -2$. $v = \text{relu}(5 - (-2)) = 7$ so $\text{mid} = 5 - 7 = -2$. Then $w = \text{relu}(\text{lo} - \text{mid}) = \text{relu}(4 + 2) = 6$ and $\text{median} = -2 + 6 = 4$.

What to notice

- **No integers needed in part 1.** max, min, abs and clamp use relu directly. Only the offset trick (part 2) and the specific ranges care about bounds.
- **Carrying has a price, and two recipes.** The pair $x = \text{relu}(x) - \text{relu}(-x)$ needs no bound on x and costs 2 neurons per value. The offset $\text{relu}(x + S) = x + S$ costs 1 neuron but only works if $x \geq -S$; a value outside the range gets clipped silently. Try $(a, b, c) = (0, -8, 3)$ with $S = 6$: $B' = \text{relu}(-8 + 6) = 0$ instead of -2, and the network returns 2 instead of the true median 0. (The file does not assert outside $[-S, S]$.)
- **Shared neurons.** h_0 in part 1 feeds three outputs. Whenever two formulas contain the same $\text{relu}(\ldots)$, they should share the neuron.
- **Depth = nesting.** max-of-4 by a chain `max(max(max(a,b),c),d)` would be 3 layers deep; by a balanced tree `max(max(a,b), max(c,d))` it is 2. Same function, same number of neurons-ish, less depth.

> **Try it**
>
> Build `max(a, b, c)` as a network. How many hidden layers, and which neurons must be carried? (Hint: $\max(a, b, c) = c + \text{relu}(\max(a, b) - c)$. Answer at the end.)

06 select: a 0/1 mask times a runtime value

`basic_python/01_foundations/06_select.py`

The problem

$$x \text{ if } m \text{ else } 0, \quad m \in \{0, 1\}, \quad x \text{ a runtime number}$$

The obvious data-flow answer is $m \cdot x$. It is exactly the thing we cannot write. In the examples so far, the values we multiplied by a mask (10 and 12 in example 02) were constants. Now x is itself something the network computes. This tiny example makes masks work on runtime values, and that unlocks everything later: loops that stop, arrays that are written, BFS frontiers.

Why `m * x` is forbidden

A layer computes $\text{relu}(Wx + b)$. Every number going into a relu is a sum of runtime values, each multiplied by a **constant** written in W. A product of two runtime values is a different kind of operation. You can see the difference geometrically: a relu network is a *piecewise linear* function of its inputs, made of flat pieces and kinks. The product $m \cdot x$ along the line $m = x$ is x^2, a parabola, which curves everywhere. No stack of relus reproduces it. (Networks can only approximate it, and we want exactness.)

We do not need the whole product, though. We need it only for $m \in \{0, 1\}$. At two values of m there is plenty of freedom to pick a piecewise linear function that matches. Big-M is one such choice.

A · control flow

```
def control_flow(m, n, x, y):
    return (n if m else 0,
            x if m else 0,
            x if m else y)
```

B · branchless arithmetic

```
def select(m, x, big_m):
    shift = big_m * (1 - m)
    return relu(x - shift) - relu(-x -
shift)

def branchless(m, n, x, y):
    return (select_nonneg(m, n, M),
            select(m, x, M),
            y + select(m, x - y, M2))
```

Stage B never writes `m * x`. `big_m * (1 - m)` is a runtime value (m) times a constant (`big_m`): allowed.

Derivation of the big-M gate

1. **Start with a non-negative x in $[0, M]$.** We want $g(1, x) = x$ and $g(0, x) = 0$. A single relu already gives the first: $\text{relu}(x) = x$. For the second, push the kink of the relu to the right: $\text{relu}(x - M) = 0$ for every $x \le M$. So the mask should shift the relu by 0 when $m = 1$ and by M when $m = 0$: the shift $M(1 - m)$ does that, and it is affine in m.

   $$\text{select}^+(m, x) = \text{relu}(x - M(1 - m)) = \text{relu}(x + Mm - M).$$

   As a neuron: weight 1 on x, weight M on m, bias $-M$. Weight times runtime input, nothing else: legal.
2. **Signed x in $[-M, M]$.** Split $x = \text{relu}(x) - \text{relu}(-x)$ and gate both halves with the same shift:

   $$\text{select}(m, x) = \text{relu}(x - M(1 - m)) - \text{relu}(-x - M(1 - m)).$$

   For $m = 1$ the shift is 0 and this is $\text{relu}(x) - \text{relu}(-x) = x$. For $m = 0$ the shift is M, so the first term is $\text{relu}(x - M)$ with $x - M \le 0$, the second is $\text{relu}(-x - M)$ with $-x - M \le 0$ (both because $|x| \le M$). Both vanish.
3. **Check the numbers** with $M = 6$:

x	−6	−5	0	3	6
$m = 1$: $\text{relu}(x) - \text{relu}(-x)$	−6	−5	0	3	6
$m = 0$: $\text{relu}(x - 6) - \text{relu}(-x - 6)$	0	0	0	0	0

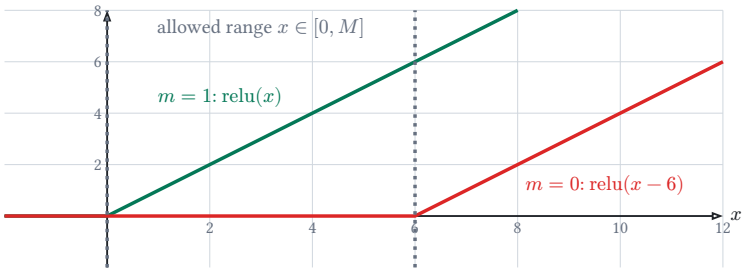


Figure 13: The mask slides the kink. For x in $[0, M]$ (between the dotted lines) the red curve ($m = 0$) is exactly zero and the green curve ($m = 1$) is the identity. The big-M rule is "the kink must move out beyond the largest x that can occur".

Two runtime values: `x if m else y`

For $m = 1$ we want x, for $m = 0$ we want y. Write it as "start at y and add the difference $x - y$ only when $m = 1$":

$$x \text{ if } m \text{ else } y = y + \text{select}(m, x - y).$$

The select now acts on $x - y$, so its bound must cover the difference: $|x - y| \le M$. With $x, y \in [-6, 6]$ that is 12, not 6. (The file calls it `M2`.) The network also needs y at the output, so $y = \text{relu}(y) - \text{relu}(-y)$ is carried by two more neurons (example 05).

An alternative needs no carrying and a tighter bound: $\text{select}(m, x) + \text{select}(1 - m, y)$. Each select only sees one value, so $M = 6$, and $1 - m$ is an affine function of m (still just a weight and a bias). Both versions are checked in the file. We use the first in stage C because it is the toolbox formula.

When M is too small

The bound is a promise: the formula is only correct when $|x| \le M$. Here is what breaks it. Take $M = 3$ and feed $x = 5$ with the mask **off** ($m = 0$):

$$\text{relu}(5 - 3) - \text{relu}(-5 - 3) = 2 - 0 = 2 \quad \text{(expected } 0\text{)}.$$

The relu was shifted by $M = 3$, but the input exceeds it, so the "off" gate still lets $x - M = 2$ through. With $x = -5$ you get -2. The mask *leaks* by $|x| - M$. With the mask on ($m = 1$) the shift is 0 and the result is always right, so tests with $m = 1$ will not catch this bug.

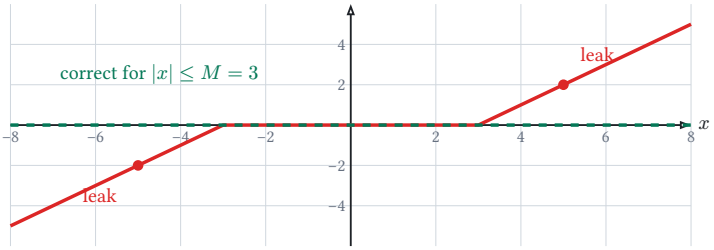


Figure 14: $\text{select}(0, x)$ with $M = 3$ (red) should be the dashed zero line. Inside $[-3, 3]$ it is. Outside it leaks $x - 3$ or $x + 3$. The dots mark $x = \pm 5$.

The same mistake in the if/else form: with $x = 6$, $y = -6$, $m = 0$ and $M = 6$ (forgetting that $|x - y|$ can be 12): $y + \text{select}(0, 12) = -6 + (\text{relu}(12 - 6) - \text{relu}(-12 - 6)) = -6 + 6 = 0$, but the answer is $y = -6$.

What about M too *big*? The arithmetic is still exact for integers, so correctness survives, but all values pass through numbers of size M. In float64, an M like 10^{16} erases the small values: for $m = 1$, $x = 1$ the neuron computes $\text{relu}(1 + 10^{16} \cdot 1 - 10^{16})$, and $1 + 10^{16}$ rounds back to 10^{16} (spacing of floats at this size is 2), giving 0 instead of 1. So pick the **tightest** bound you can prove. Derive it, don't guess:

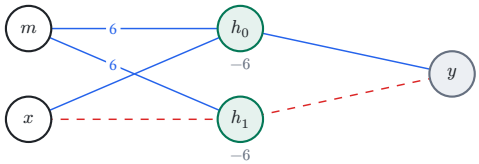
1. if x is an input in $[-6, 6]$: $M = 6$;
2. if it is a difference of two such values: $M = 12$;
3. if it is a sum of values with bounds M_1, M_2: $M = M_1 + M_2$ (bounds add); a value multiplied by a constant k gets $M \cdot |k|$;
4. if it is itself the output of a select: it lies in $[-M, M]$ (or 0), the same M as before.

Stage C: the network

The smallest piece is the signed select, m and x in, two neurons, one output:

$$W_1 = \begin{pmatrix} 6 & 1 \\ 6 & -1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} -6 \\ -6 \end{pmatrix}$$

$$W_2 = (1 \ \ -1), \quad b_2 = (0)$$



The file stacks all three variants into one network with four inputs (m, n, x, y) and seven hidden neurons, where $n \in [0, 6]$ is the non-negative case. $M = 6$ and $M_2 = 12$:

$$W_1 = \begin{pmatrix} 6 & 1 & 0 & 0 \\ 6 & 0 & 1 & 0 \\ 6 & 0 & -1 & 0 \\ 12 & 0 & 1 & -1 \\ 12 & 0 & -1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} -6 \\ -6 \\ -6 \\ -12 \\ -12 \\ 0 \\ 0 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 1 & -1 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

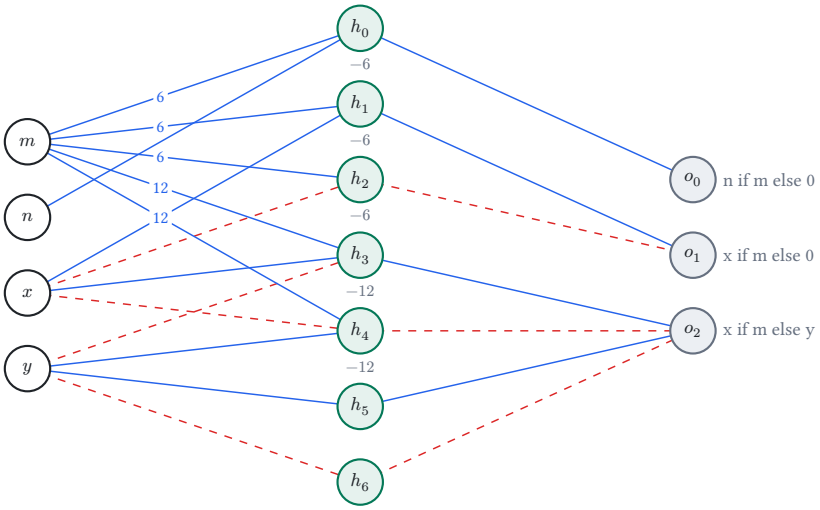


Figure 15: h_0: non-negative select of n. (h_1, h_2): signed select of x. (h_3, h_4): signed select of $x - y$ with $M_2 = 12$. (h_5, h_6): carry y. The mask m reaches five neurons through its weights 6 or 12.

Worked trace for $(m, n, x, y) = (1, 4, -5, 2)$ and for the same values with $m = 0$:

	$m = 1$	$m = 0$
$W_1 v + b_1$	$(4, -5, 5, -7, 7, 2, -2)$	$(-2, -11, -1, -19, -5, 2, -2)$
$h = \text{relu}(\ldots)$	$(4, 0, 5, 0, 7, 2, 0)$	$(0, 0, 0, 0, 0, 2, 0)$
$o_0 = h_0$	4	0
$o_1 = h_1 - h_2$	$0 - 5 = -5$	0
$o_2 = h_3 - h_4 + h_5 - h_6$	$0 - 7 + 2 - 0 = -5$	$0 - 0 + 2 - 0 = 2$

With the mask on: $o = (4, -5, -5)$, that is, n, x and x. With the mask off: $o = (0, 0, 2)$, that is, 0, 0 and y.

What to notice

- **Masks shift thresholds.** The mask never multiplies anything. It moves the point at which each relu turns on, far out of range when $m = 0$. This one idea is the whole mechanism for "doing something only if…" in every later chapter.

- **The mask lands in the weights.** Weight M on the m input, bias $-M$. If m is not an input but the result of an earlier layer, the same weights sit on the previous layer's output. The mask must be an exact 0/1 number, which is why example 01 insisted on integers.
- **Cost:** non-negative select 1 neuron; signed select 2; two-value if/else 4 (2 + 2 for carrying y). M is a bound you have to prove, not tune.
- **Pitfall: only half the cases catch a bad M.** A too-small M is invisible when $m = 1$. Always test the mask-off case at the extremes of the range.
- **Pitfall: bounds on differences and sums.** $|x - y|$ can be twice as big as $|x|$. Compute M for the expression inside the select.

> **Try it**
>
> Write `x if m else 100` for $x \in [0, 50]$ as a network with as few hidden neurons as you can. Which value must be carried, and what is M? (Answer below.)

Answers to the exercises

01 · `x >= 3`. Shift the threshold: $\text{step}(x-2) = \text{relu}(x-2) - \text{relu}(x-3)$. Only the first-layer bias changes: $b_1 = (-2,-3)$; $W_1 = (1,1)^T$, $W_2 = (1,-1)$, $b_2 = 0$. Check $x=2$: hidden $(0,0)$, output 0. $x=3$: hidden $(1,0)$, output 1. $x=5$: hidden $(3,2)$, output 1.

02 · `1 if a < 0 else 5 if a < 10 else 9`. Start at the first branch's value and add a jump at each boundary. The jump from 1 to 5 happens at $a \geq 0$ (that is $\text{step}(a+1)$). The jump from 5 to 9 happens at $a \geq 10$ (that is $\text{step}(a-9)$):

$$y = 1 + 4\,\text{step}(a+1) + 4\,\text{step}(a-9) = 1 + 4(\text{relu}(a+1) - \text{relu}(a)) + 4(\text{relu}(a-9) - \text{relu}(a-10)).$$

Four hidden neurons, biases $(1, 0, -9, -10)$, output weights $(4, -4, 4, -4)$, output bias 1. Check: $a=-1$: all four are 0, $y=1$. $a=0$: $(1,0,0,0)$, $y=5$. $a=10$: $(11,10,1,0)$, $y = 1+4+4 = 9$.

03 · `abs(a - b) <= 2`. With $d = a-b$, "$d \geq -2$" is $\text{step}(d+3)$ and "$d \leq 2$" is $\text{step}(3-d)$. They are never both 0, so the AND is just their sum minus 1: $y = \text{step}(d+3) + \text{step}(3-d) - 1$. Four neurons: $\text{relu}(d+3), \text{relu}(d+2), \text{relu}(3-d), \text{relu}(2-d)$, output $h_0 - h_1 + h_2 - h_3 - 1$. Check: $d=0$: $3-2+3-2-1 = 1$; $d=3$: $(6-5)+(0-0)-1 = 0$; $d=-3$: $(0-0)+(6-5)-1 = 0$.

04 · majority of three. $s = b_0 + b_1 + b_2$. "At least 2" is $s \geq 2$, i.e. $\text{step}(s-1) = \text{relu}(s-1) - \text{relu}(s-2)$: two hidden neurons, each with weights $(1,1,1)$ and biases -1 and -2. For "at least 3 of 5": five inputs, weights $(1,1,1,1,1)$, $\text{step}(s-2) = \text{relu}(s-2) - \text{relu}(s-3)$, so biases $-2, -3$.

05 · max of three. $\text{hi} = \max(a,b) = b + u$ with $u = \text{relu}(a-b)$, and $\max(a,b,c) = c + \text{relu}(\text{hi} - c)$. Layer 1: $u = \text{relu}(a-b)$, $B' = \text{relu}(b+S)$, $C' = \text{relu}(c+S)$. Layer 2: $v = \text{relu}(B' + u - C')$ and $C'' = \text{relu}(C')$ (the carried copy of c). Output: $y = C'' + v - S$. Two hidden layers (two nested relus), 3 + 2 neurons. Carried: b (once, into layer 2) and c (through both layers). Check $(a,b,c) = (1,5,3)$ with $S=6$: $u=0$, $B'=11$, $C'=9$; $v = \text{relu}(11+0-9) = 2$, $C''=9$; $y = 9+2-6 = 5$.

06 · `x if m else 100`, $x \in [0, 50]$. The else-branch is a constant, so it is affine in m: x if m else $100 = \text{relu}(x - 50(1-m)) + 100(1-m)$. The relu is the non-negative select with $M = 50$. But the output layer needs m itself to form $100(1-m)$, and $m \geq 0$, so carry it with one more neuron. Two hidden neurons: $h_0 = \text{relu}(x + 50m - 50)$, $h_1 = \text{relu}(m)$. Output: $y = h_0 - 100h_1 + 100$. Check $x=30$: $m=1$: $h=(30,1)$, $y = 30 - 100 + 100 = 30$. $m=0$: $h=(0,0)$, $y=100$.